

step02_01_ETL_cleaning

October 5, 2026

```
[33]: import duckdb
import json
import pandas as pd
from pathlib import Path

pd.set_option("display.width", 220)
pd.set_option("display.max_columns", 50)
pd.set_option("display.max_rows", 120)

BASE_DIR = Path.cwd().parent
DATA_DIR = BASE_DIR / "data_sets"
RESULT_DIR = BASE_DIR / "results"
RESULT_DIR.mkdir(exist_ok=True)
TMP_DIR = RESULT_DIR / "duckdb_tmp"
TMP_DIR.mkdir(exist_ok=True)

con = duckdb.connect(str(RESULT_DIR / "step01_03_5.duckdb"))
con.execute("SET memory_limit='6GB'") # lower this if your laptop
    ↳ has < 8 GB RAM
con.execute(f"SET temp_directory='{TMP_DIR.as_posix()}'"")
con.execute("SET preserve_insertion_order=false")

FINDINGS = {} # everything that matters is collected here -> saved as
    ↳ JSON at the end

def q(sql):
    return con.execute(sql).df()

def save(df, name):
    df.to_csv(RESULT_DIR / name, index=False)
    print(f" -> saved results/{name}")

[34]: # ## 0. Load tables into DuckDB (delay is loaded once and cached in the .duckdb
    ↳ file)
```

```

def load(name, fname, types=None):
    path = (DATA_DIR / fname).as_posix()
    t = f", types={types}" if types else ""
    con.execute(
        f"CREATE OR REPLACE TABLE {name} AS "
        f"SELECT * FROM read_csv('{path}', header=true{t})"
    )

load("train_details", "train_details.csv", {"train_no": "BIGINT"})
load("station_full_names", "station_full_names.csv")
load("schedule", "combined_schedule.csv", {"train_no": "BIGINT",
↪ "station_no": "INTEGER"})

already = con.execute(
    "SELECT count(*) FROM information_schema.tables WHERE table_name='delay'"
).fetchone()[0]
if not already:
    print("Loading combined_delay.csv (one-time, a few minutes)...")
    load("delay", "combined_delay.csv",
        {"date": "DATE", "station_no": "INTEGER", "delay": "DOUBLE",
↪ "train_no": "BIGINT"})

for t in ["train_details", "station_full_names", "schedule", "delay"]:
    print(t, con.execute(f"SELECT count(*) FROM {t}").fetchone()[0])

# Helper tables used repeatedly
con.execute("CREATE OR REPLACE TABLE sched_stations AS SELECT DISTINCT
↪ station_name FROM schedule")
con.execute("CREATE OR REPLACE TABLE delay_stations AS SELECT DISTINCT
↪ station_name FROM delay")

```

```

train_details 8992
station_full_names 8963
schedule 172112
delay 38428703

```

[34]: <_duckdb.DuckDBPyConnection at 0x15db5b2bb70>

```

[35]: #####
      # Handling missing station addresses
      #####

      # Define the file path
      file_path = (RESULT_DIR / 'dim_station_resolved.csv').as_posix()
      # Load the resolved station dimension dataset
      dim_station = pd.read_csv(file_path)

```

```

# Replace the 94 null station_address fields with the global constant "UNKNOWN"
dim_station['station_address'] = dim_station['station_address'].
    ↪fillna('UNKNOWN')

# Verify the missing values have been successfully resolved
remaining_nulls = dim_station['station_address'].isnull().sum()
print(f"Remaining null station addresses: {remaining_nulls}")

# Save the cleaned dataset back to the results directory
dim_station.to_csv(file_path, index=False)

```

Remaining null station addresses: 0

```

[36]: #_
    ↪=====
# VERIFICATION of missing arrival time at source and missing departure time at_
    ↪destination
#_
    ↪=====

# Define the file path
input_file = DATA_DIR / 'combined_schedule.csv'
output_file = RESULT_DIR / 'combined_schedule_imputed.csv'

# Load the schedule dataset
schedule = pd.read_csv(input_file)
# Calculate the maximum station_no (destination) for each train
schedule['max_station_no'] = schedule.groupby('train_no')['station_no'].
    ↪transform('max')

# Isolate missing condition masks
missing_arrival = schedule['arrival_time'].isnull()
missing_departure = schedule['departure_time'].isnull()

# Verify arrival times
arr_at_origin = (missing_arrival & (schedule['station_no'] == 1)).sum()
arr_not_origin = (missing_arrival & (schedule['station_no'] != 1)).sum()

# Verify departure times
dep_at_dest = (missing_departure & (schedule['station_no'] ==_
    ↪schedule['max_station_no'])).sum()
dep_not_dest = (missing_departure & (schedule['station_no'] !=_
    ↪schedule['max_station_no'])).sum()

print("--- VERIFICATION RESULTS ---")

```

```

print(f"Missing arrival times exactly at origin (station_no = 1): {arr_at_origin}")
print(f"Missing arrival times elsewhere: {arr_not_origin}")
print(f"Missing departure times exactly at destination: {dep_at_dest}")
print(f"Missing departure times elsewhere: {dep_not_dest}")

```

--- VERIFICATION RESULTS ---

```

Missing arrival times exactly at origin (station_no = 1): 8673
Missing arrival times elsewhere: 0
Missing departure times exactly at destination: 8673
Missing departure times elsewhere: 0

```

```

[37]: # =====
# APPLY IMPUTATION RULE
# =====
# Verification proves the missing values are structural -> apply the Zero-Halt
# Rule.
# Fill missing origin arrival times with their scheduled departure time
schedule.loc[(missing_arrival) & (schedule['station_no'] == 1), 'arrival_time']
# = schedule['departure_time']

# Fill missing destination departure times with their scheduled arrival time
schedule.loc[(missing_departure) & (schedule['station_no'] ==
# schedule['max_station_no']), 'departure_time'] = schedule['arrival_time']

# Verify the final state
remaining_arr_nulls = schedule['arrival_time'].isnull().sum()
remaining_dep_nulls = schedule['departure_time'].isnull().sum()

print("\n--- IMPUTATION RESULTS ---")
print(f"Remaining missing arrival times overall: {remaining_arr_nulls}")
print(f"Remaining missing departure times overall: {remaining_dep_nulls}")

# Drop the helper column and save the cleaned dataset

schedule = schedule.drop(columns=['max_station_no'])
schedule.to_csv(output_file, index=False)
print(f"\n-> Saved cleaned schedule to {output_file.as_posix()}")

```

--- IMPUTATION RESULTS ---

```

Remaining missing arrival times overall: 0
Remaining missing departure times overall: 0

```

```

-> Saved cleaned schedule to D:/M_Tech_CSE_AI/SEM01/DWDM/project/indian_railway_
delay_LOCK/results/combined_schedule_imputed.csv

```

```

[48]: # =====
# VERIFY ARRIVAL VS DEPARTURE DELAY
# =====
print("\n=== Delay Type Verification ===")
# this is important for correctly handling missing delay values

# Execute the verification query out-of-core, explicitly filtering out missing
↳delays
verification_query = """
WITH TrainDestinations AS (
    SELECT train_no, MAX(station_no) AS max_station_no
    FROM schedule
    GROUP BY train_no
)
SELECT
    COUNT(CASE WHEN d.station_no = 1 AND d.delay > 0 THEN 1 END) AS
↳delayed_at_origin,
    COUNT(CASE WHEN d.station_no = t.max_station_no AND d.delay > 0 THEN 1 END)
↳AS delayed_at_destination
FROM delay d
JOIN TrainDestinations t ON d.train_no = t.train_no
WHERE d.delay IS NOT NULL
"""

results = con.execute(verification_query).df()

# Extract the scalar counts from the dataframe
delayed_at_origin = int(results['delayed_at_origin'][0])
delayed_at_dest = int(results['delayed_at_destination'][0])

print("--- DELAY FIELD VERIFICATION (EXCLUDING MISSING VALUES) ---")
print(f"Non-zero delays at origin (station_no = 1): {delayed_at_origin}")
print(f"Non-zero delays at destination: {delayed_at_dest}\n")

print("--- INTERPRETATION ---")
# Implement the conditional logic based on the defined hypotheses
if delayed_at_origin > 0 and delayed_at_dest > 0:
    # Result: Both hypotheses are TRUE.
    print("Conclusion: Departure delay is recorded at the source station, and
↳Arrival delay is recorded at intermediate and destination stations.")

elif delayed_at_origin > 0 and delayed_at_dest == 0:
    # Result: Hypothesis 1 is TRUE and Hypothesis 2 is FALSE.
    print("Conclusion: The delay field represents Departure delay for each
↳station and train.")

elif delayed_at_origin == 0 and delayed_at_dest > 0:

```

```

    # Result: Hypothesis 1 is FALSE and Hypothesis 2 is TRUE.
    print("Conclusion: The delay field represents Arrival delay for each
    ↪station and train.")

else:
    # Result: Both hypotheses are FALSE.
    print("Conclusion: We will assume Arrival delay at each station, and for
    ↪the destination it is departure delay. There is data loss at both the origin
    ↪and destination.")

```

=== Delay Type Verification ===

--- DELAY FIELD VERIFICATION (EXCLUDING MISSING VALUES) ---

Non-zero delays at origin (station_no = 1): 1038047

Non-zero delays at destination: 1044006

--- INTERPRETATION ---

Conclusion: Departure delay is recorded at the source station, and Arrival delay is recorded at intermediate and destination stations.

```

[49]: # Formulate the finding based on the query results
FINDINGS["delay_field_resolution"] = {
    "hypothesis_tested": "Composite Context Switch",
    "delayed_at_origin_count": delayed_at_origin,
    "delayed_at_destination_count": delayed_at_dest,
    "conclusion": "Hypothesis confirmed. The delay field is context-dependent.",
    "etl_rule": {
        "station_no_1": "Strictly Departure Delay",
        "station_no_greater_than_1": "Strictly Arrival Delay"
    }
}

# Save this to a new findings file for Phase 1 (ETL)
etl_findings_path = RESULT_DIR / "step02_01_ETL_FINDINGS.json"

with open(etl_findings_path, "w") as f:
    json.dump(FINDINGS, f, indent=4)

print(f"-> Saved delay context findings to {etl_findings_path.as_posix()}")

```

-> Saved delay context findings to D:/M_Tech_CSE_AI/SEM01/DWDM/project/indian_railway_delay_LOCK/results/step02_01_ETL_FINDINGS.json

[]: